

WIPRO CAPSTONE PROJECT

Code Implementation Report

BlazeDemo Flight Booking Automation Framework

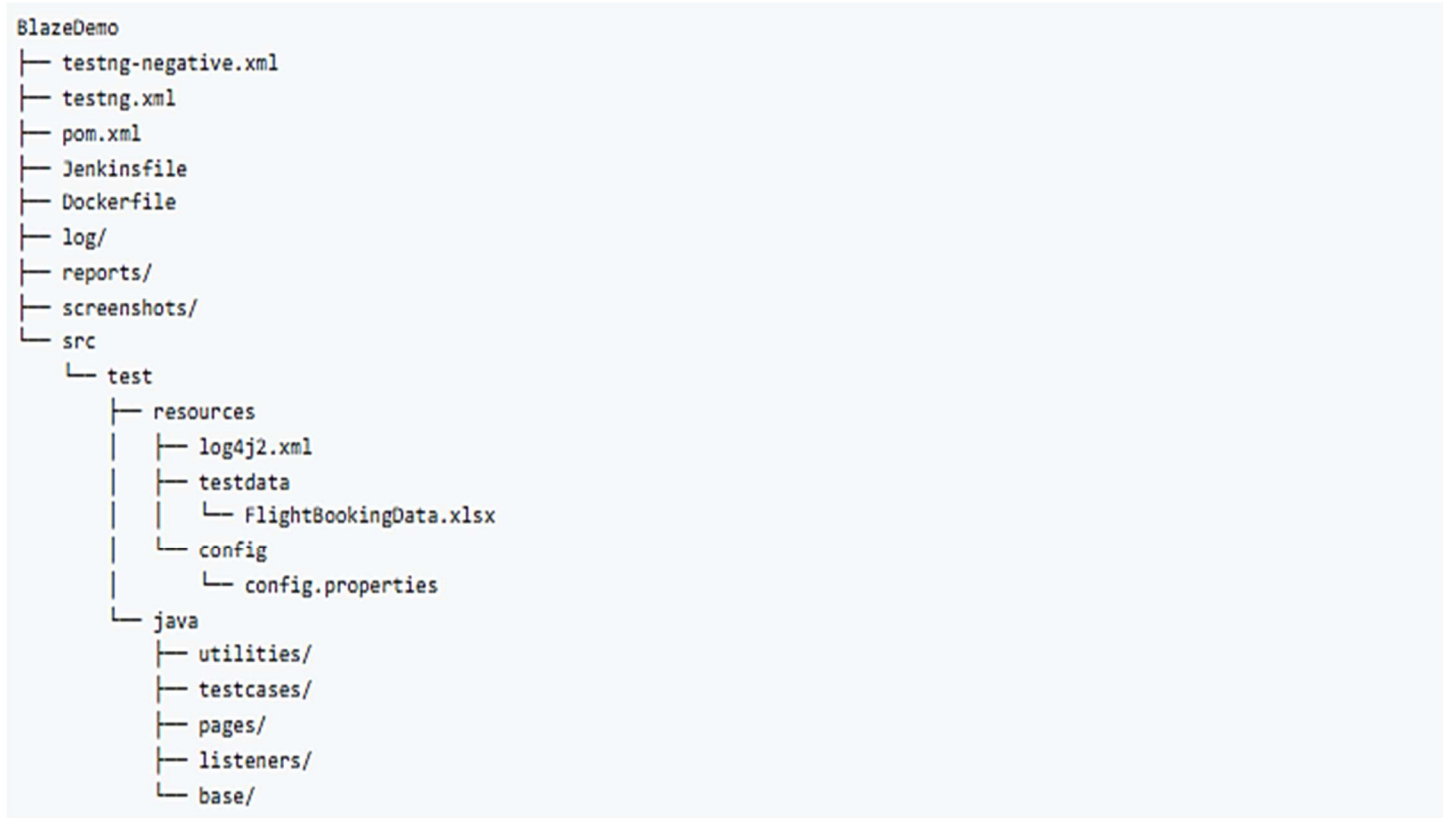
Selenium WebDriver · TestNG · Maven · ExtentReports · Jenkins · Docker

Project Name	BlazeDemo Flight Booking Automation Framework
Document Type	Code Implementation Report
Prepared By	Mr. Baivab Sarkar
Mentor/ Trainer	Mrs. Vaishali Sonanis
Application Under Test	BlazeDemo
URL	https://blazedemo.com/
Repository	ThisIs-Developer/Wipro-Capstone-Project
GitHub Link	https://github.com/ThisIs-Developer/Wipro-Capstone-Project
Date	13 th June 2026

1. Introduction

This document provides the complete source code implementation details for the BlazeDemo Automation Framework. It includes the framework architecture, configuration files, utility classes, page objects, test classes, listeners, CI/CD configuration, and supporting resources used for automated testing.

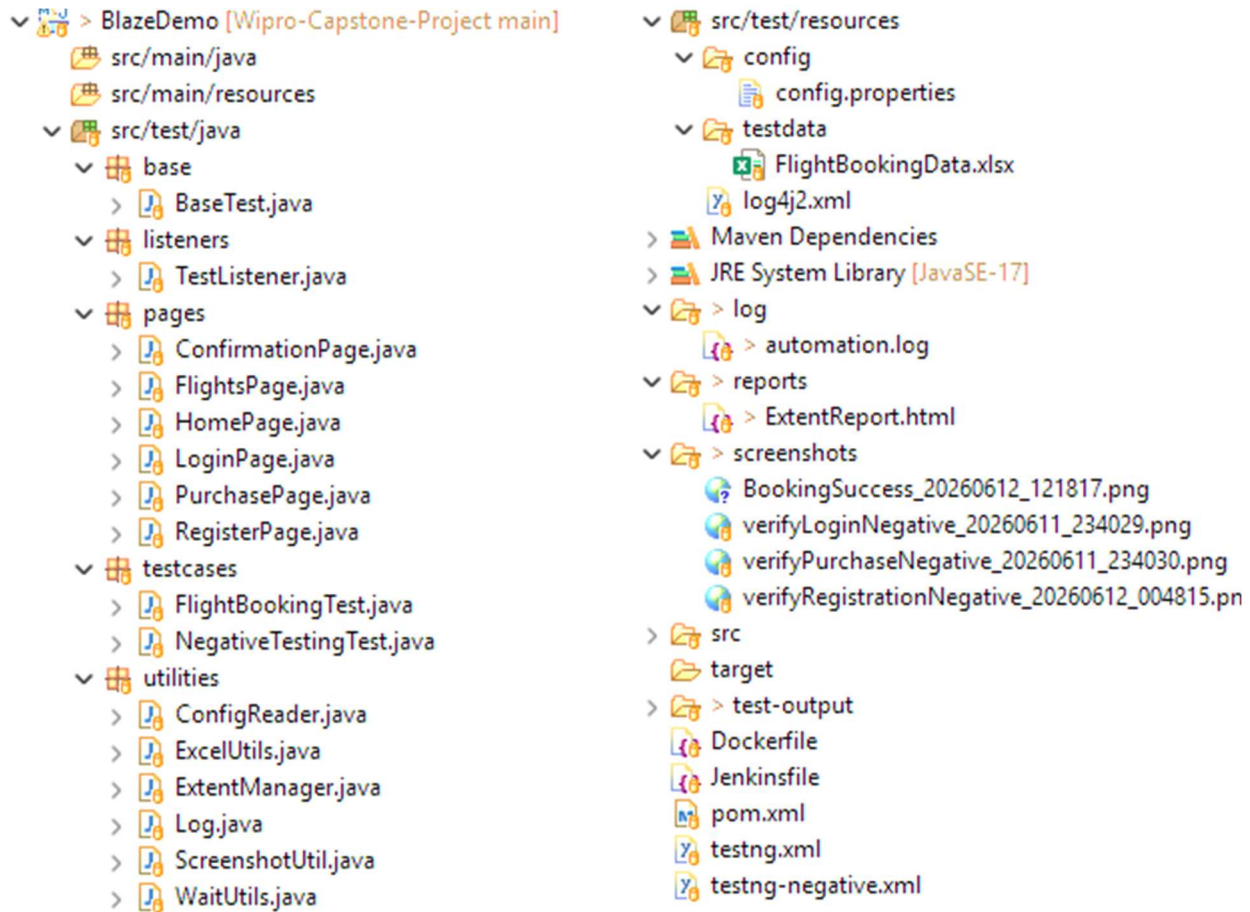
2. Framework Structure



The project is structured around the Page Object Model (POM) design pattern, which separates page interaction logic from test logic. This ensures that any UI change requires updates in only one place, the corresponding page class, rather than across all test scripts

Layer	Package	Files	Responsibility
Foundation	base	BaseTest.java	WebDriver lifecycle management — setup, config, teardown
Listening	listeners	TestListener.java	TestNG event hooks — logging, screenshot on failure
Pages (POM)	pages	6 Page classes	Element locators & interaction methods per application page
Test Cases	testcases	FlightBookingTest, NegativeTestingTest	Test logic — assertions, data-driven execution, reporting
Utilities	utilities	6 helper classes	Reusable services — config, Excel, waits, screenshots, logging
Config	resources	config.properties, testdata XLSX	Externalised test configuration and data
CI/CD	root	Jenkinsfile, Dockerfile	Automated build, test, and container deployment pipeline

3. Project Structure in IDE



4. Source code

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>BlazeDemo</groupId>
  <artifactId>BlazeDemo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.44.0</version>
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.12.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

```

</dependency>
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.5.1</version>
</dependency>
<dependency>
  <groupId>commons-io</groupId>
  <artifactId>commons-io</artifactId>
  <version>2.22.0</version>
</dependency>
<dependency>
  <groupId>com.aventstack</groupId>
  <artifactId>extentreports</artifactId>
  <version>5.1.2</version>
</dependency>
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-api</artifactId>
  <version>2.26.0</version>
</dependency>
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.26.0</version>
</dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.15.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.5.4</version>
      <configuration>
        <suiteXmlFiles>
          <suiteXmlFile>testng.xml</suiteXmlFile>
        </suiteXmlFiles>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

config.properties

```

browser=chrome
url=https://blazedemo.com/

```

log4j2.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<Configuration status="WARN">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %c{1} - %msg%n"/>
    </Console>
    <File name="FileLogger"
      fileName="log/automation.log"
      append="false">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %c{1} - %msg%n"/>
    </File>
  </Appenders>
  <Loggers>
    <Root level="info">
      <AppenderRef ref="Console"/>
      <AppenderRef ref="FileLogger"/>
    </Root>
  </Loggers>
</Configuration>

```

testng.xml

```

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="FlightBookingSuite">
  <listeners>
    <listener class-name="listeners.TestListener"/>
  </listeners>
  <!-- <test name="SmokeTests">
    <groups><run><include name="smoke"/></run>
    </groups>
    <classes>
      <class name="testcases.FlightBookingTest"/>
    </classes>
  </test>-->
  <test name="FlightBookingTests" group-by-instances="true">
    <classes>
      <class name="testcases.FlightBookingTest"/>
    </classes>
  </test>
</suite>

```

testng-negative.xml

```

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="NegativeTestSuite">
  <listeners>
    <listener class-name="listeners.TestListener"/>
  </listeners>
  <test name="NegativeTests">
    <classes>
      <class name="testcases.NegativeTestingTest"/>
    </classes>
  </test>
</suite>

```

Jenkinsfile

```

pipeline {
  agent any
  tools {
    maven 'Maven3'
  }
}

```

```

}

stages {
    stage('Checkout') {
        steps {
            git branch: 'main',
                url: 'https://github.com/ThisIs-Developer/Wipro-Capstone-Project'
        }
    }

    stage('Build') {
        steps {
            dir('BlazeDemo') {
                bat 'mvn clean compile'
            }
        }
    }

    stage('Compile') {
        steps {
            dir('BlazeDemo') {
                bat 'mvn compile'
            }
        }
    }

    stage('Test') {
        steps {
            dir('BlazeDemo') {
                bat 'mvn test'
            }
        }
    }

    stage('Docker Build') {
        steps {
            dir('BlazeDemo') {
                bat 'docker build -t blazedemo-framework .'
            }
        }
    }

    stage('Docker Run') {
        steps {
            bat 'docker run --rm blazedemo-framework'
        }
    }
}

post {
    success {
        echo 'Build Successful'
    }

    failure {
        echo 'Build Failed'
    }

    always {
        echo 'Pipeline Execution Completed'
    }
}

```

```

    }
}
}

```

Dockerfile

```

FROM selenium/standalone-chrome:latest
USER root
WORKDIR /app
RUN apt-get update && apt-get install -y maven openjdk-25-jdk
COPY . .
RUN mvn clean install -DskipTests
CMD ["mvn", "test", "-DsuiteXmlFile=testng.xml"]

```

BaseTest.java

```

package base;

import java.time.Duration;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;

import utilities.ConfigReader;
import utilities.Log;

public class BaseTest {

    public WebDriver driver;

    @BeforeClass
    public void setup() {

        String browser = ConfigReader.getProperty("browser");

        switch (browser.toLowerCase()) {
            case "chrome":
                ChromeOptions options = new ChromeOptions();

                options.addArguments("--headless=new");
                options.addArguments("--no-sandbox");
                options.addArguments("--disable-dev-shm-usage");
                options.addArguments("--disable-gpu");
                options.addArguments("--remote-allow-origins=*");

                driver = new ChromeDriver(options);

                Log.info("Launching Browser : " + browser);
                break;

            case "firefox":
                driver = new FirefoxDriver();
                Log.info("Launching Browser : " + browser);
                break;
        }
    }
}

```

```

        case "edge":
            driver = new EdgeDriver();
            Log.info("Launching Browser : " + browser);
            break;

        default:
            Log.error("Unsupported Browser : " + browser);
            throw new RuntimeException("Unsupported Browser : " + browser);
    }

    driver.manage().window().maximize();
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
    driver.get(ConfigReader.getProperty("url"));
    Log.info("Application Opened");
}

@AfterClass
public void tearDown() {
    if (driver != null) {
        Log.info("Closing Browser");
        driver.quit();
    }
}
}

```

HomePage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.ui.Select;

import utilities.WaitUtils;

public class HomePage {

    private WebDriver driver;
    private WaitUtils wait;

    public HomePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WaitUtils(driver);
    }

    private By departureDropdown = By.name("fromPort");
    private By destinationDropdown = By.name("toPort");
    private By findFlightsButton = By.cssSelector("input[type='submit']");

    public String getPageTitle() {
        return driver.getTitle();
    }

    public String getPageUrl() {
        return driver.getCurrentUrl();
    }
}

```

```

public boolean isDepartureDropdownDisplayed() {
    WebElement element = driver.findElement(departureDropdown);
    wait.waitForVisibility(element);
    return element.isDisplayed();
}

public boolean isDestinationDropdownDisplayed() {
    WebElement element = driver.findElement(destinationDropdown);
    wait.waitForVisibility(element);
    return element.isDisplayed();
}

public boolean isFindFlightsButtonDisplayed() {
    WebElement element = driver.findElement(findFlightsButton);
    wait.waitForVisibility(element);
    return element.isDisplayed();
}

public void selectDepartureCity(String city) {
    Select select = new Select(driver.findElement(departureDropdown));
    select.selectByVisibleText(city);
}

public void selectDestinationCity(String city) {
    Select select = new Select(driver.findElement(destinationDropdown));
    select.selectByVisibleText(city);
}

public void clickFindFlights() {
    WebElement button = driver.findElement(findFlightsButton);
    wait.waitForClickable(button);
    button.click();
}
}

```

FlightsPage.java

```

package pages;

import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import utilities.WaitUtils;

public class FlightsPage {

    private WebDriver driver;
    private WaitUtils wait;

    private By flightTable = By.cssSelector("table[class='table']");
    private By flightRows = By.xpath("/html/body/div[2]/table/tbody/tr");
    private By chooseFlightButtons = By.cssSelector("input[value='Choose This Flight']");

    public FlightsPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WaitUtils(driver);
    }
}

```

```

public boolean isFlightTableDisplayed() {

    WebElement table = driver.findElement(flightTable);
    wait.waitForVisibility(table);
    return table.isDisplayed();
}

public int getFlightCount() {
    List<WebElement> rows = driver.findElements(flightRows);
    //    System.out.println("Total Flight Count: "+rows.size());
    return rows.size();
}

public boolean hasFlights() {
    return getFlightCount() > 0;
}

public void chooseFirstFlight() {
    List<WebElement> buttons = driver.findElements(chooseFlightButtons);
    if (buttons.isEmpty()) {
        throw new RuntimeException("No Flight Found");
    }
    wait.waitForClickable(buttons.get(0));
    buttons.get(0).click();
}
}

```

PurchasePage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.ui.Select;

import utilities.WaitUtils;

public class PurchasePage {

    private WebDriver driver;
    private WaitUtils wait;

    private By priceText = By.xpath("/html/body/div[2]/p[3]");
    private By totalCost = By.xpath("/html/body/div[2]/p[4]");
    private By purchaseFlightButton = By.cssSelector("input[value='Purchase Flight']");
    private By nameField = By.id("inputName");
    private By addressField = By.id("address");
    private By cityField = By.id("city");
    private By stateField = By.id("state");
    private By zipCodeField = By.id("zipCode");
    private By cardTypeDropdown = By.id("cardType");
    private By cardNumberField = By.id("creditCardNumber");
    private By monthField = By.id("creditCardMonth");
    private By yearField = By.id("creditCardYear");
    private By nameOnCardField = By.id("nameOnCard");

    public PurchasePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WaitUtils(driver);
    }
}

```

```

}

public boolean isPriceDisplayed() {
    WebElement price = driver.findElement(priceText);
    wait.waitForVisibility(price);
    return price.isDisplayed();
}

public boolean isTotalCostDisplayed() {
    WebElement cost = driver.findElement(totalCost);
    wait.waitForVisibility(cost);
    return cost.isDisplayed();
}

public void enterName(String name) {
    driver.findElement(nameField).sendKeys(name);
}

public void enterAddress(String address) {
    driver.findElement(addressField).sendKeys(address);
}

public void enterCity(String city) {
    driver.findElement(cityField).sendKeys(city);
}

public void enterState(String state) {
    driver.findElement(stateField).sendKeys(state);
}

public void enterZipCode(String zipCode) {
    driver.findElement(zipCodeField).sendKeys(zipCode);
}

public void selectCardType(String cardType) {
    Select select = new Select(driver.findElement(cardTypeDropdown));
    select.selectByVisibleText(cardType);
}

public void enterCardNumber(String cardNumber) {
    driver.findElement(cardNumberField).sendKeys(cardNumber);
}

public void enterMonth(String month) {
    driver.findElement(monthField).clear();
    driver.findElement(monthField).sendKeys(month);
}

public void enterYear(String year) {
    driver.findElement(yearField).clear();
    driver.findElement(yearField).sendKeys(year);
}

public void enterNameOnCard(String nameOnCard) {
    driver.findElement(nameOnCardField).sendKeys(nameOnCard);
}

public void clickPurchaseFlight() {
    WebElement button = driver.findElement(purchaseFlightButton);
    wait.waitForClickable(button);
    button.click();
}

```

```
}  
}
```

ConfirmationPage.java

```
package pages;  
  
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.WebElement;  
  
import utilities.WaitUtils;  
  
public class ConfirmationPage {  
  
    private WebDriver driver;  
    private WaitUtils wait;  
  
    private By successMessage = By.tagName("h1");  
    private By purchaseId = By.xpath("/html/body/div[2]/div/table/tbody/tr[1]/td[2]");  
    private By status = By.xpath("/html/body/div[2]/div/table/tbody/tr[2]/td[2]");  
    private By amount = By.xpath("/html/body/div[2]/div/table/tbody/tr[3]/td[2]");  
    private By cardNumber = By.xpath("/html/body/div[2]/div/table/tbody/tr[4]/td[2]");  
    private By date = By.xpath("/html/body/div[2]/div/table/tbody/tr[7]/td[2]");  
  
    public ConfirmationPage(WebDriver driver) {  
        this.driver = driver;  
        this.wait = new WaitUtils(driver);  
    }  
  
    public String getSuccessMessage() {  
        WebElement element = driver.findElement(successMessage);  
        wait.waitForVisibility(element);  
        return element.getText();  
    }  
  
    public String getPurchaseId() {  
        return driver.findElement(purchaseId).getText();  
    }  
  
    public String getStatus() {  
        return driver.findElement(status).getText();  
    }  
  
    public String getAmount() {  
        return driver.findElement(amount).getText();  
    }  
  
    public String getCardNumber() {  
        return driver.findElement(cardNumber).getText();  
    }  
  
    public String getDate() {  
        return driver.findElement(date).getText();  
    }  
}
```

LoginPage.java

```
package pages;
```

```

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {

    WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    private By email = By.id("email");
    private By password = By.id("password");
    private By loginButton = By.xpath("//button[contains(normalize-space(),'Login')]");

    public void enterEmail(String emailId) {
        driver.findElement(email).sendKeys(emailId);
    }

    public void enterPassword(String pwd) {
        driver.findElement(password).sendKeys(pwd);
    }

    public void clickLogin() {
        driver.findElement(loginButton).click();
    }

    public void login(String emailId, String pwd) {
        enterEmail(emailId);
        enterPassword(pwd);
        clickLogin();
    }

    public boolean is419ErrorDisplayed() {
        return driver.getPageSource().contains("419") || driver.getPageSource().contains("Page
Expired");
    }
}

```

RegisterPage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class RegisterPage {

    WebDriver driver;

    public RegisterPage(WebDriver driver) {
        this.driver = driver;
    }

    private By name = By.id("name");
    private By company = By.id("company");
    private By email = By.id("email");
    private By password = By.id("password");
    private By confirmPassword = By.id("password-confirm");

```

```

        private By registerButton = By.xpath("//button[contains(normalize-space(),'Register')]");

        public void registerUser(String userName, String companyName, String emailId, String pwd,
String confirmPwd) {
            driver.findElement(name).sendKeys(userName);
            driver.findElement(company).sendKeys(companyName);
            driver.findElement(email).sendKeys(emailId);
            driver.findElement(password).sendKeys(pwd);
            driver.findElement(confirmPassword).sendKeys(confirmPwd);
            driver.findElement(registerButton).click();
        }

        public boolean is419ErrorDisplayed() {
            return driver.getPageSource().contains("419") || driver.getPageSource().contains("Page
Expired");
        }
    }
}

```

ConfigReader.java

```

package utilities;

import java.io.FileInputStream;
import java.util.Properties;

public class ConfigReader {

    private static Properties properties;

    static {
        try {
            properties = new Properties();
            FileInputStream fis = new
FileInputStream("src/test/resources/config/config.properties");
            properties.load(fis);
        } catch (Exception e) {
            throw new RuntimeException("Unable to load config file", e);
        }
    }

    public static String getProperty(String key) {
        return properties.getProperty(key);
    }
}

```

ExcelUtils.java

```

package utilities;

import java.io.FileInputStream;

import org.apache.poi.ss.usermodel.DataFormatter;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class ExcelUtils {

    public static Object[][] getExcelData(String filePath, String sheetName) {
        try (FileInputStream fis = new FileInputStream(filePath);
            XSSFWorkbook workbook = new XSSFWorkbook(fis)) {

```

```

        XSSFSheet sheet = workbook.getSheet(sheetName);
        int rowCount = sheet.getLastRowNum();
        int cellCount = sheet.getRow(0).getLastCellNum();

        Object[][] data = new Object[rowCount][cellCount];
        DataFormatter formatter = new DataFormatter();
        for (int i = 1; i <= rowCount; i++) {
            for (int j = 0; j < cellCount; j++) {
                data[i - 1][j] = formatter.formatCellValue(sheet.getRow(i).getCell(j));
            }
        }
        return data;
    } catch (Exception e) {
        throw new RuntimeException("Unable to read excel data", e);
    }
}
}

```

WaitUtils.java

```

package utilities;

import java.time.Duration;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class WaitUtils {

    private WebDriverWait wait;

    public WaitUtils(WebDriver driver) {
        try {
            wait = new WebDriverWait(driver, Duration.ofSeconds(20));
        } catch (Exception e) {
            throw new RuntimeException("Unable to initialize WaitUtils", e);
        }
    }

    public void waitForVisibility(WebElement element) {
        try {
            wait.until(ExpectedConditions.visibilityOf(element));
        } catch (Exception e) {
            throw new RuntimeException("Element not visible", e);
        }
    }

    public void waitForClickable(WebElement element) {
        try {
            wait.until(ExpectedConditions.elementToBeClickable(element));
        } catch (Exception e) {
            throw new RuntimeException("Element not clickable", e);
        }
    }

    public void waitForTitle(String title) {

```

```

        try {
            wait.until(ExpectedConditions.titleContains(title));
        } catch (Exception e) {
            throw new RuntimeException("Title not found : " + title, e);
        }
    }

    public void waitForUrl(String url) {

        try {
            wait.until(ExpectedConditions.urlContains(url));
        } catch (Exception e) {
            throw new RuntimeException("URL not found : " + url, e);
        }
    }
}

```

ScreenshotUtil.java

```

package utilities;

import java.io.File;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

import org.apache.commons.io.FileUtils;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public class ScreenshotUtil {

    public static String captureScreenshot(WebDriver driver, String testName) {

        try {
            String timestamp
=LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss"));
            String filePath = "screenshots/"+ testName+ "_" + timestamp+ ".png";

            File source =((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
            File destination = new File(filePath);
            FileUtils.copyFile(source,destination);
            return filePath;
        } catch (IOException e) {
            throw new RuntimeException("Unable to capture screenshot", e);
        }
    }
}

```

ExtentManager.java

```

package utilities;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;

public class ExtentManager {

    private static ExtentReports extent;

```

```

private ExtentManager() {}

public static ExtentReports getInstance() {
    if (extent == null) {
        ExtentSparkReporter sparkReporter = new ExtentSparkReporter(
"reports/ExtentReport.html");
        sparkReporter.config().setDocumentTitle("Flight Booking Automation Report");
        sparkReporter.config().setReportName( "Automation Execution Results");

        extent = new ExtentReports();
        extent.attachReporter(sparkReporter);
    }
    return extent;
}
}

```

Log.java

```

package utilities;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class Log {

    private static final Logger logger = LogManager.getLogger(Log.class);
    private Log() {}
    public static void info(String message) {
        logger.info(message);
    }
    public static void error(String message) {
        logger.error(message);
    }
}

```

TestListener.java

```

package listeners;

import org.testng.ITestListener;
import org.testng.ITestResult;

import base.BaseTest;
import utilities.Log;
import utilities.ScreenshotUtil;

public class TestListener implements ITestListener {

    @Override
    public void onTestStart(ITestResult result) {
        System.out.println("STARTED : "+ result.getName());
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        System.out.println("PASSED : "+ result.getName());
    }

    @Override
    public void onTestFailure(ITestResult result) {

```

```

        System.out.println("FAILED : "+ result.getName());
    try {
        BaseTest testClass = (BaseTest) result.getInstance();
        String screenshotPath =
ScreenshotUtil.captureScreenshot(testClass.driver,result.getName());
        Log.error("Test Failed : "+ result.getName());
        Log.error("Failure Screenshot Saved : "+ screenshotPath);
    } catch (Exception e) {
        Log.error("Unable to capture screenshot : "+ e.getMessage());
    }
}

@Override
public void onTestSkipped(ITestResult result) {
    System.out.println("SKIPPED : "+ result.getName());
}
}

```

FlightBookingTest.java

```

package testcases;

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Factory;
import org.testng.annotations.Test;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;

import base.BaseTest;
import pages.ConfirmationPage;
import pages.FlightsPage;
import pages.HomePage;
import pages.PurchasePage;
import utilities.ExcelUtils;
import utilities.ExtentManager;
import utilities.Log;
import utilities.ScreenshotUtil;

public class FlightBookingTest extends BaseTest {

    private final ExtentReports extent = ExtentManager.getInstance();
    private ExtentTest test;

    private HomePage homePage;
    private FlightsPage flightsPage;
    private PurchasePage purchasePage;
    private ConfirmationPage confirmationPage;

    private String departureCity;
    private String destinationCity;

    private String name;
    private String address;
    private String city;
    private String state;
    private String zipCode;

    private String cardType;

```

```

private String cardNumber;
private String month;
private String year;
private String nameOnCard;

@DataProvider(name = "bookingData")
public static Object[][] bookingData() {
    return
ExcelUtils.getExcelData("src/test/resources/testdata/FlightBookingData.xlsx", "BookingData");
}

@Factory(dataProvider = "bookingData")
public FlightBookingTest(String departureCity, String destinationCity, String name, String
address, String city, String state,
    String zipCode, String cardType, String cardNumber, String month, String year, String
nameOnCard) {
    this.departureCity = departureCity;
    this.destinationCity = destinationCity;
    this.name = name;
    this.address = address;
    this.city = city;
    this.state = state;
    this.zipCode = zipCode;
    this.cardType = cardType;
    this.cardNumber = cardNumber;
    this.month = month;
    this.year = year;
    this.nameOnCard = nameOnCard;
}

@Test(priority=1, groups={"smoke", "regression"})
public void verifyHomePage() {
    test = extent.createTest(departureCity+"->"+"destinationCity");
    Log.info("Home Page Verification Started");

    homePage = new HomePage(driver);
    Assert.assertEquals(homePage.getPageTitle(), "BlazeDemo");
    Assert.assertEquals(homePage.getPageUrl(), "https://blazedemo.com/");
    Assert.assertTrue(homePage.isDepartureDropdownDisplayed());
    Assert.assertTrue(homePage.isDestinationDropdownDisplayed());
    Assert.assertTrue(homePage.isFindFlightsButtonDisplayed());
    homePage.selectDepartureCity(departureCity);
    homePage.selectDestinationCity(destinationCity);
    homePage.clickFindFlights();

    test.pass("Home Page Verified Successfully");
    Log.info("Home Page Verification Completed");
}

@Test(priority = 2, groups = {"regression"}, dependsOnMethods = "verifyHomePage")
public void verifyFlightSearch() {
    test = extent.createTest("Verify Flight Search");
    Log.info("Flight Search Verification Started");

    flightsPage = new FlightsPage(driver);
    Assert.assertTrue(flightsPage.isFlightTableDisplayed());
    Assert.assertTrue(flightsPage.hasFlights());
    flightsPage.chooseFirstFlight();

    test.pass("Flight Search Verified");
    Log.info("Flight Search Verification Completed");
}

```

```

}

@Test(priority = 3, groups = { "regression" }, dependsOnMethods = "verifyFlightSearch")
public void verifyPurchasePage() {
    test = extent.createTest("Verify Purchase Page");
    Log.info("Purchase Page Verification Started");

    purchasePage = new PurchasePage(driver);
    Assert.assertTrue(purchasePage.isPriceDisplayed());
    Assert.assertTrue(purchasePage.isTotalCostDisplayed());
    purchasePage.enterName(name);
    purchasePage.enterAddress(address);
    purchasePage.enterCity(city);
    purchasePage.enterState(state);
    purchasePage.enterZipCode(zipCode);
    purchasePage.selectCardType(cardType);
    purchasePage.enterCardNumber(cardNumber);
    purchasePage.enterMonth(month);
    purchasePage.enterYear(year);
    purchasePage.enterNameOnCard(nameOnCard);
    purchasePage.clickPurchaseFlight();

    test.pass("Passenger And Payment Details Submitted");
    Log.info("Purchase Page Verification Completed");
}

@Test(priority = 4, groups = { "regression" }, dependsOnMethods = "verifyPurchasePage")
public void verifyBookingConfirmation() throws Exception {
    test = extent.createTest("Verify Booking Confirmation");
    Log.info("Confirmation Verification Started");

    confirmationPage = new ConfirmationPage(driver);
    Assert.assertEquals(confirmationPage.getSuccessMessage(), "Thank you for your purchase
today!");
    Assert.assertFalse(confirmationPage.getPurchaseId().isEmpty());
    Assert.assertEquals(confirmationPage.getStatus(), "PendingCapture");
    Assert.assertFalse(confirmationPage.getAmount().isEmpty());
    Assert.assertFalse(confirmationPage.getCardNumber().isEmpty());
    Assert.assertFalse(confirmationPage.getDate().isEmpty());

    String screenshotPath = ScreenshotUtil.captureScreenshot(driver, "BookingSuccess");

    test.addScreenCaptureFromPath(screenshotPath);
    Log.info("Screenshot Captured");
    test.pass("Booking Confirmation Verified");
    Log.info("Booking Confirmation Completed");
    extent.flush();
}
}

```

NegativeTestingTest.java

```

package testcases;

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.ConfirmationPage;
import pages.LoginPage;

```

```

import pages.PurchasePage;
import pages.RegisterPage;
import utilities.ConfigReader;

public class NegativeTestingTest extends BaseTest {

    private RegisterPage registerPage;
    private LoginPage loginPage;
    private PurchasePage purchasePage;
    private ConfirmationPage confirmationPage;

    @DataProvider(name = "userData")
    public Object[][] userData() {
        return new Object[][] {
            {"Baivab Sarkar", "Wipro", "baivab@test.com", "Test@123", "Test@123"}
        };
    }

    @DataProvider(name = "purchaseNegativeData")
    public Object[][] purchaseNegativeData() {
        return new Object[][] {
            {"", "123456", "@#$%^&*", "98765", "ABCDEF", "Visa", "1", "99", "1900", "@@##"}
        };
    }

    @Test(priority = 1, dataProvider = "userData")
    public void verifyRegistrationNegative(String name, String company, String email, String password, String confirmPassword){
        driver.get(ConfigReader.getProperty("url")+"register");
        registerPage = new RegisterPage(driver);

        registerPage.registerUser(name, company, email, password, confirmPassword);
        Assert.assertTrue(registerPage.is419ErrorDisplayed(),"Known Defect: Registration request results in HTTP 419 Page Expired error.");
    }

    @Test(priority = 2, dataProvider = "userData")
    public void verifyLoginNegative(String name, String company, String email, String password, String confirmPassword){
        driver.get(ConfigReader.getProperty("url") + "login");
        loginPage = new LoginPage(driver);

        loginPage.login(email,password);
        Assert.assertTrue(loginPage.is419ErrorDisplayed(),"Known Defect: Login request results in HTTP 419 Page Expired error.");
    }

    @Test(priority = 3, dataProvider = "purchaseNegativeData")
    public void verifyPurchaseNegative(String name, String address, String city, String state, String zipCode, String cardType, String cardNumber, String month, String year, String nameOnCard){

        driver.get(ConfigReader.getProperty("url") + "purchase.php");
        purchasePage = new PurchasePage(driver);

        purchasePage.enterName(name);
        purchasePage.enterAddress(address);
        purchasePage.enterCity(city);
        purchasePage.enterState(state);
        purchasePage.enterZipCode(zipCode);
        purchasePage.selectCardType(cardType);
    }
}

```

```

purchasePage.enterCardNumber(cardNumber);
purchasePage.enterMonth(month);
purchasePage.enterYear(year);
purchasePage.enterNameOnCard(nameOnCard);
purchasePage.clickPurchaseFlight();

confirmationPage = new ConfirmationPage(driver);

Assert.assertEquals(
    confirmationPage.getSuccessMessage(),
    "Thank you for your purchase today!",
    "Critical Defect: System generated a booking confirmation despite invalid
passenger and payment information.");

Assert.assertFalse(
    confirmationPage.getPurchaseId().isEmpty(),
    "Critical Defect: Booking ID was generated for invalid booking data.");

Assert.assertEquals(
    confirmationPage.getStatus(),
    "PendingCapture",
    "Critical Defect: Invalid booking received a valid transaction status.");

Assert.assertFalse(
    confirmationPage.getAmount().isEmpty(),
    "Critical Defect: Amount was generated despite invalid input data.");

Assert.assertFalse(
    confirmationPage.getCardNumber().isEmpty(),
    "Critical Defect: Invalid payment information was accepted.");

Assert.assertFalse(
    confirmationPage.getDate().isEmpty(),
    "Critical Defect: Confirmation record was created for invalid booking data.");
}
}

```

FlightBookingData.xlsx

	A	B	C	D	E	F	G	H	I	J	K	L
1	DepartureCity	DestinationCity	Name	Address	City	State	ZipCode	CardType	CardNumber	Month	Year	NameOnCard
2	Boston	London	Baivab Sarkar	Park Street Area	Kolkata	West Bengal	700001	Diner's Club	4111111111111110	12	2028	Baivab Sarkar
3	Paris	Rome	Ayush Sharma	Park Circus	Howrah	West Bengal	745501	Visa	378282246310005	10	2029	Ayush Sharma
4	Mexico City	Dublin	Harsh Choudhary	College Street	DumDum	West Bengal	798457	American Express	30569309025904	9	2030	Harsh Choudhary